Intelligence artificielle et apprentissage automatique

Informatique — Licence 3 (L3)

Cours INF-L3-07

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Comprendre et pratiquer le machine learning : histoire et domaines de l'IA, apprentissage supervisé (régression, classification) et non supervisé (clustering), réseaux de neurones, initiation au traitement du langage, évaluation rigoureuse des modèles et projet Python complet avec scikit-learn.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Histoire et grands domaines de l'intelligence artificielle
• Leçon 2 — Apprentissage supervisé : régression et classification
• Leçon 3 — Apprentissage non supervisé : le clustering
• Leçon 4 — Réseaux de neurones : principes de fonctionnement
• Leçon 5 — Traitement automatique du langage naturel : initiation
• Leçon 6 — Évaluer un modèle : exactitude, précision et rappel
• Leçon 7 — Projet : construire et évaluer un modèle simple en Python
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

L'intelligence artificielle est partout : elle filtre tes messages indésirables, recommande des vidéos, reconnaît les visages sur les photos et répond, depuis peu, à des questions en langage naturel. Mais derrière le mot « IA », il n'y a ni magie ni conscience : il y a des mathématiques, des données et des programmes qui apprennent à partir d'exemples. Ce cours te donne les clés pour passer du statut d'utilisateur impressionné à celui de praticien capable d'entraîner, d'évaluer et de critiquer un modèle.

La méthode dominante aujourd'hui est l'apprentissage automatique (machine learning) : au lieu d'écrire à la main les règles d'une tâche — impossible pour reconnaître un chat sur une photo — on montre à un algorithme des milliers d'exemples et on le laisse ajuster ses propres paramètres. Nous suivrons la chaîne complète : d'abord l'histoire et les grands domaines, puis l'apprentissage supervisé et non supervisé, les réseaux de neurones, le traitement du langage, et enfin l'évaluation, étape trop souvent négligée alors qu'elle sépare un modèle sérieux d'une belle illusion.

Ce cours prolonge directement « Introduction à la data science » (INF-L2-25) et « Statistiques et analyse de données » (INF-L2-13) : tu y retrouveras Python, pandas et la démarche train/test. Tous les exemples sont de petits jeux de données fabriqués pour le cours et réellement exécutés avec la bibliothèque scikit-learn ; chaque chiffre annoncé peut être reproduit en recopiant le code. C'est une exigence : en IA, on ne croit un résultat que s'il est mesuré, jamais s'il est seulement raconté.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Raconter les grandes étapes de l'histoire de l'IA et situer l'apprentissage automatique parmi ses domaines.
- Distinguer régression et classification, et entraîner un modèle supervisé simple avec scikit-learn.
- Expliquer et appliquer l'algorithme K-Means pour regrouper des données non étiquetées.
- Calculer à la main la sortie d'un neurone artificiel (somme pondérée, biais, fonction d'activation).
- Transformer un texte en vecteur numérique par la méthode du sac de mots.
- Calculer et interpréter exactitude, précision, rappel et score F1 à partir d'une matrice de confusion.
- Mener un mini-projet complet : préparer les données, entraîner, évaluer et critiquer honnêtement un modèle.

Leçon 1 — Histoire et grands domaines de l'intelligence artificielle

L'intelligence artificielle naît officiellement en 1956, lors de l'atelier de Dartmouth (aux États-Unis), où John McCarthy propose le terme « intelligence artificielle » et réunit les pionniers du domaine. Quelques années plus tôt, en 1950, Alan Turing avait déjà posé la question « les machines peuvent-elles penser ? » et proposé son célèbre test d'imitation. L'optimisme des débuts est immense : en 1957-1958, Frank Rosenblatt présente le perceptron, un premier modèle de neurone artificiel capable d'apprendre à classer des formes simples, et les journaux annoncent des machines pensantes pour bientôt.

La réalité rattrape vite ces promesses : les machines des années 1960 sont trop faibles et les problèmes réels trop complexes. Le financement se tarit : c'est le premier « hiver de l'IA » (milieu des années 1970 au début des années 1980). Le domaine rebondit avec les systèmes experts — des programmes qui enchaînent des règles fournies par des spécialistes humains, comme MYCIN pour le diagnostic médical — avant un second hiver à la fin des années 1980, ces systèmes se révélant coûteux et fragiles. Entre-temps, en 1986, la popularisation de l'algorithme de rétropropagation relance discrètement les réseaux de neurones.

Le tournant vient de trois facteurs combinés : des données massives (le Web), des calculs parallèles puissants (les cartes graphiques) et des algorithmes mûris. Jalons : en 1997, Deep Blue d'IBM bat le champion du monde d'échecs Garry Kasparov ; en 2011, Watson gagne au jeu télévisé Jeopardy! ; en 2012, le réseau profond AlexNet écrase la compétition de reconnaissance d'images ImageNet (environ 15 % d'erreur contre 26 % pour le second) ; en 2016, AlphaGo bat Lee Sedol au jeu de go ; en 2017, l'architecture Transformer ouvre la voie aux grands modèles de langage actuels. Les grands domaines de l'IA se lisent dans ces succès : vision par ordinateur, traitement du langage, robotique, systèmes de recommandation, aide à la décision — tous alimentés

aujourd'hui par l'apprentissage automatique, auquel ce cours est consacré.

L'essentiel
Née en 1956 à Dartmouth, l'IA a connu deux « hivers » avant d'exploser grâce au trio données massives + puissance de calcul + algorithmes d'apprentissage. Aujourd'hui, le machine learning est son moteur principal, présent dans la vision, le langage, la recommandation et la décision.

Exemple — Deux époques, deux philosophies : règles écrites à la main contre apprentissage

```
ÉPOQUE 1 — LE SYSTÈME EXPERT (années 1980)
SI température > 38,5 ET toux = oui ET courbatures = oui
ALORS suspecter une grippe
→ Les règles sont écrites par des humains, une à une.
→ Le système ne sait RIEN en dehors de ses règles.

ÉPOQUE 2 — L'APPRENTISSAGE AUTOMATIQUE (aujourd'hui)
On donne 50 000 dossiers de patients étiquetés
(« grippe » ou « pas grippe ») à un algorithme.
L'algorithme ajuste lui-même des paramètres internes
pour séparer au mieux les deux groupes.
→ Les « règles » sont APPRISES à partir des exemples.
→ Le système peut traiter des cas jamais vus…
à condition qu'ils ressemblent aux exemples appris.

LA FRISE À RETENIR
1950 Test de Turing
1956 Atelier de Dartmouth : le mot « IA » est créé
1957 Perceptron de Rosenblatt (premier neurone qui apprend)
1974 Début du premier hiver de l'IA (financements coupés)
1980 Essor des systèmes experts… puis second hiver (~1987)
1986 La rétropropagation relance les réseaux de neurones
1997 Deep Blue bat Kasparov aux échecs
2011 Watson gagne à Jeopardy!
2012 AlexNet : le deep learning gagne ImageNet
2016 AlphaGo bat Lee Sedol au jeu de go
2017 L'architecture Transformer (base des modèles de langage)
```

À retenir

- 1956, Dartmouth : naissance officielle de l'IA ; 1950, Turing avait déjà posé la question fondatrice.
- Deux hivers de l'IA montrent le danger des promesses excessives : un domaine se juge à ses résultats mesurés.
- Le deep learning triomphe après 2012 grâce aux données massives et aux cartes graphiques, pas grâce à une seule idée géniale.

Exercice

Classe ces événements dans l'ordre chronologique et associe chacun à un domaine de l'IA (vision, langage, jeu, décision) : (a) AlexNet remporte ImageNet ; (b) atelier de Dartmouth ; (c) Watson gagne à Jeopardy! ; (d) perceptron de Rosenblatt ; (e) AlphaGo

bat Lee Sedol. Puis explique en deux phrases pourquoi l'année 2012 est considérée comme un tournant alors que le perceptron date de 1957.

✔ Corrigé
Ordre : (b) 1956 → (d) 1957 → (c) 2011 → (a) 2012 → (e) 2016. Domaines : Dartmouth fonde le domaine entier ; le perceptron relève de la reconnaissance de formes (vision) ; Watson, du langage et de la question-réponse ; AlexNet, de la vision par ordinateur ; AlphaGo, du jeu et de la décision séquentielle. Pourquoi 2012 est un tournant : le perceptron de 1957 était limité à une seule couche de neurones, entraîné sur de minuscules données avec des machines très lentes. En 2012, AlexNet combine réseaux profonds (plusieurs couches), plus d'un million d'images d'entraînement et la puissance des cartes graphiques : c'est la recette données + calcul + algorithme qui manquait depuis un demi-siècle, et son avance énorme sur les concurrents (environ 15 % d'erreur contre 26 %) l'a prouvée d'un coup.

Leçon 2 — Apprentissage supervisé : régression et classification

L'apprentissage supervisé ressemble à un élève qui révise avec un corrigé : on lui donne des exemples accompagnés de la bonne réponse, appelée étiquette (ou label). À partir de ces exemples, l'algorithme apprend une fonction qui, pour un nouvel exemple jamais vu, prédit l'étiquette. On distingue deux grandes familles selon la nature de la réponse à prédire. Si la réponse est un nombre continu — un prix, une température, une note — c'est une régression. Si la réponse est une catégorie — admis ou ajourné, pourriel ou normal, chat ou chien — c'est une classification.

La régression linéaire est le modèle de régression le plus simple : elle ajuste une droite (en une dimension) qui passe au plus près des points d'exemple, en minimisant l'erreur quadratique moyenne. L'exemple ci-dessous prédit la note d'un étudiant selon ses heures de révision. Le résultat obtenu en l'exécutant est une droite d'équation note ≈ 1,405 × heures + 6,429, avec un coefficient R^2 de 0,993 : les points fabriqués pour ce cours sont presque parfaitement alignés, ce qui est rare dans la vraie vie — sur des données réelles, un R^2 entre 0,5 et 0,8 est déjà honorable. Le R^2 mesure la part de variation expliquée par le modèle : 1 serait parfait, 0 signifierait que le modèle ne fait pas mieux que prédire la moyenne.

Pour la classification, l'exemple utilise une régression logistique — malgré son nom, c'est un classifieur — qui prédit l'admission d'un étudiant à partir de ses heures de révision et de son assiduité. Entraîné sur 8 étudiants et testé sur 4 autres, le modèle obtient une exactitude de 0,75 : il se trompe sur un étudiant très assidu (80 %) mais qui révisait peu (3 h), classé admis à tort. Deux leçons à retenir. D'abord, on n'évalue JAMAIS un modèle sur les données qui ont servi à l'entraîner : le jeu de test doit être gardé de côté, sinon c'est comme noter un élève sur les exercices du cours déjà corrigés. Ensuite, une exactitude calculée sur 4 exemples vaut ce qu'elle vaut : un seul exemple vaut 25 points de pourcentage. Nous y reviendrons à la leçon 6.

L'essentiel
Supervisé = apprendre à partir d'exemples étiquetés. Cible continue → régression (on mesure l'erreur, par exemple avec R^2) ; cible catégorielle → classification (on mesure l'exactitude sur un jeu de test séparé). Le jeu de test ne doit jamais servir à l'entraînement.

Exemple — Régression linéaire puis classification logistique (scikit-learn, résultats réels)

```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# --- Régression : prédire une note selon les heures de révision ---
heures = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
notes = np.array([8, 9, 11, 12, 13, 15, 16, 18])
reg = LinearRegression().fit(heures, notes)
print("Équation : note ≈", round(reg.coef_[0], 3), "× heures +",
round(reg.intercept_, 3)) # note ≈ 1.405 × heures + 6.429
print("R² =", round(reg.score(heures, notes), 3)) # 0.993
print("Prédiction pour 5,5 h :", round(reg.predict([[5.5]])[0], 2)) # 14.15

# --- Classification : prédire l'admission (1) ou l'ajournement (0) ---
# Colonnes : heures de révision, assiduité en %
X = np.array([[2, 40], [3, 55], [4, 60], [5, 70], [6, 65], [7, 80],
[8, 85], [1, 30], [9, 90], [5, 45], [6, 90], [3, 80]])
y = np.array([0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0])
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=4, random_state=0)
clf = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
print("Exactitude sur le test :", accuracy_score(y_te, clf.predict(X_te)))
# -> 0.75 (3 étudiants sur 4 bien classés)
print("Nouvel étudiant [5 h, 75 %] ->", clf.predict([[5, 75]])[0]) # 1 (admis)
print("Nouvel étudiant [2 h, 35 %] ->", clf.predict([[2, 35]])[0]) # 0
```

À retenir

- Régression = prédire un nombre ; classification = prédire une catégorie. Le type de la cible décide de la famille.
- fit() entraîne, predict() prédit, score() évalue : c'est le cycle de tous les modèles scikit-learn.
- Un R^2 très élevé sur un exemple fabriqué ne prouve rien sur le terrain : seules comptent les performances sur des données nouvelles.

Exercice

1) Avec la droite de l'exemple (note ≈ 1,405 × heures + 6,429), calcule la note prédite pour 9 heures de révision. Le résultat est-il cohérent avec l'échelle des notes sur 20 ? Quelle prudence faut-il avoir quand on prédit au-delà des données observées (ici, 1 à 8 heures) ? 2) Pour chacun de ces problèmes, dis s'il s'agit d'une régression ou d'une classification : prédire le prix d'un téléphone d'occasion ; prédire si un message est un pourriel ; prédire la consommation d'essence d'un véhicule ; prédire si une transaction

bancaire est frauduleuse.

✔ Corrigé
1) Note prédite = 1,405 × 9 + 6,429 = 12,645 + 6,429 = 19,07 (valeur confirmée par l'exécution du modèle). Le résultat reste sous 20, mais il faut être prudent : 9 heures est en dehors de l'intervalle observé (1 à 8 h) — on parle d'extrapolation. La relation peut cesser d'être linéaire (la fatigue fait plafonner les progrès, et la droite finirait par dépasser 20, ce qui est absurde). Plus on s'éloigne des données d'entraînement, moins la prédiction est digne de confiance. 2) Prix d'un téléphone : régression (un montant continu). Pourriel ou non : classification (deux catégories). Consommation d'essence : régression (des litres aux 100 km). Transaction frauduleuse ou non : classification. Réflexe général : demande-toi « la réponse est-elle un nombre sur une échelle continue, ou un nom de catégorie ? »

Leçon 3 — Apprentissage non supervisé : le clustering

Dans l'apprentissage non supervisé, il n'y a pas de corrigé : les données arrivent sans étiquettes, et l'algorithme doit y découvrir lui-même une structure. La tâche la plus connue est le clustering (regroupement) : rassembler les exemples qui se ressemblent en groupes appelés clusters, de sorte que deux exemples d'un même groupe soient proches et deux exemples de groupes différents soient éloignés. Une boutique qui classe ses clients en « petits acheteurs », « clients réguliers » et « gros clients » sans l'avoir décidé à l'avance fait du clustering.

L'algorithme de clustering le plus utilisé est K-Means. Son principe tient en quatre étapes : on choisit le nombre de groupes K et on place K points de départ, les centres ; chaque exemple est rattaché au centre le plus proche ; chaque centre est déplacé au barycentre (la moyenne) des exemples qui lui sont rattachés ; on répète affectation et déplacement jusqu'à ce que plus rien ne bouge. K-Means minimise ce qu'on appelle l'inertie : la somme des distances au carré entre chaque point et son centre. Plus l'inertie est faible, plus les groupes sont compacts — mais elle diminue toujours quand K augmente, donc elle ne dit pas à elle seule quel K choisir.

L'exemple exécuté ci-dessous regroupe 12 clients d'une boutique décrits par deux variables : visites par mois et panier moyen. Avec K = 3, l'algorithme retrouve exactement la structure cachée dans ces données fabriquées : un groupe centré sur (1,5 visite ; 7,25 milliers de FCFA) — les petits acheteurs ; un groupe centré sur (4,5 ; 21,25) — les réguliers ; un groupe centré sur (8,25 ; 43,25) — les gros clients, avec une inertie finale de 133,0. Deux limites à connaître : K doit être choisi par l'humain (l'algorithme ne le devine pas), et le résultat dépend des centres initiaux — c'est pourquoi on lance l'algorithme plusieurs fois (paramètre n_init) et on garde la meilleure solution. Enfin, les numéros de clusters (0, 1, 2) n'ont aucun sens en eux-mêmes : c'est à l'analyste de nommer les groupes en regardant leurs centres.

L'essentiel
Non supervisé = pas d'étiquettes, l'algorithme cherche une structure. K-Means alterne affectation au centre le plus proche et déplacement des centres au barycentre, jusqu'à stabilisation. K est choisi par l'humain ; les noms des groupes aussi.

Exemple — K-Means sur 12 clients d'une boutique (centres réels obtenus par exécution)

```
import numpy as np
from sklearn.cluster import KMeans

# Clients : (visites par mois, panier moyen en milliers de FCFA)
clients = np.array([[1, 5], [2, 8], [1, 10], [2, 6],
[8, 40], [9, 45], [7, 38], [9, 50],
[4, 20], [5, 22], [4, 25], [5, 18]])
km = KMeans(n_clusters=3, random_state=0, n_init=10).fit(clients)
print("Étiquettes :", km.labels_.tolist())
# -> [1, 1, 1, 1, 0, 0, 0, 0, 2, 2, 2, 2]
print("Centres des groupes :")
for i, c in enumerate(km.cluster_centers_):
print(" groupe", i, ": visites ≈", round(c[0], 2),
"| panier ≈", round(c[1], 2), "milliers de FCFA")
# -> groupe 0 : (8.25 ; 43.25) les gros clients
# -> groupe 1 : (1.5 ; 7.25) les petits acheteurs
# -> groupe 2 : (4.5 ; 21.25) les clients réguliers
print("Inertie :", round(km.inertia_, 1)) # 133.0
```

À retenir

- K-Means : affecter chaque point au centre le plus proche, recalculer les centres, répéter jusqu'à convergence.
- Le nombre K de groupes est un choix humain, éclairé par l'inertie et par le bon sens métier — pas une vérité automatique.
- Un numéro de cluster n'est pas une étiquette : nommer les groupes fait partie du travail d'analyse.

Exercice

Quatre points sont donnés : A(1, 1), B(2, 1), C(8, 9), D(9, 8). On lance K-Means avec K = 2 en prenant comme centres initiaux A et D. Fais tourner l'algorithme à la main : 1) rattache chaque point au centre le plus proche ; 2) recalcule les deux centres (moyennes des points rattachés) ; 3) refais une affectation et conclus : l'algorithme a-t-il convergé ? Donne les groupes finaux et leurs centres.

✔ Corrigé
Itération 1 — affectation (distances euclidiennes) : A est à distance 0 du centre A et loin de D → groupe 1. B(2, 1) : distance 1 à A, environ 9,9 à D → groupe 1. C(8, 9) : environ 11,4 à A, environ 1,4 à D → groupe 2. D → groupe 2. Nouveaux centres : groupe 1 = moyenne de A et B = ((1+2)/2 ; (1+1)/2) = (1,5 ; 1,0) ; groupe 2 = moyenne de C et D = ((8+9)/2 ; (9+8)/2) = (8,5 ; 8,5). Itération 2 : avec ces centres, A et B restent nettement plus proches de (1,5 ; 1,0), C et D de (8,5 ; 8,5) : aucune affectation ne change, les centres ne bougent donc plus — convergence atteinte (résultat confirmé en exécutant le calcul). Groupes finaux : {A, B} centré en (1,5 ; 1,0) et {C, D} centré en (8,5 ; 8,5).

Leçon 4 — Réseaux de neurones : principes de fonctionnement

Un réseau de neurones artificiels est un modèle composé d'unités de calcul très simples, les neurones, organisées en couches. Chaque neurone reçoit plusieurs valeurs en entrée, les multiplie chacune par un poids, additionne le tout, ajoute un biais, puis transforme cette somme par une fonction d'activation. Les poids et le biais sont les paramètres du modèle : ce sont eux que l'entraînement ajuste, exactement comme la pente et l'ordonnée à l'origine de la régression linéaire, mais par millions. Le perceptron de 1957 était un unique neurone ; les réseaux modernes en empilent des couches entières — d'où le terme « apprentissage profond » (deep learning) quand il y a beaucoup de couches.

Le calcul d'un neurone tient en une formule : $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$, puis sortie = f(z). Les poids mesurent l'importance de chaque entrée : un poids positif amplifie, un poids négatif inhibe, un poids nul rend l'entrée invisible. Les fonctions d'activation les plus connues sont la sigmoïde, qui écrase z dans l'intervalle (0 ; 1) — interprétable comme une probabilité —, et ReLU, qui vaut 0 si z est négatif et z sinon. Sans fonction d'activation non linéaire, empiler des couches ne servirait à rien : la composition de fonctions linéaires reste linéaire, et le réseau ne pourrait pas apprendre de motifs complexes comme les formes dans une image.

Dans l'exemple exécuté ci-dessous, un neurone reçoit trois entrées (0,8 ; 0,5 ; 0,3) avec les poids (0,4 ; −0,2 ; 0,9) et un biais de −0,1. Les produits valent 0,32, −0,10 et 0,27 ; la somme pondérée donne z = 0,39 ; la sigmoïde transforme ce score en 0,5963, soit une « probabilité » d'environ 60 % si ce neurone devait décider d'une classe. Avec d'autres entrées (0,1 ; 0,9 ; 0,2), le même neurone donne z = −0,06 : ReLU l'éteint complètement (sortie 0) tandis que la sigmoïde répond 0,485, juste sous le seuil de décision 0,5. L'entraînement d'un réseau consiste à ajuster tous ces poids, exemple après exemple, en suivant la pente de l'erreur — c'est la rétropropagation, qui propage le « reproche » de la sortie vers les couches précédentes. À ton niveau, retenir le calcul d'un neurone et le rôle de chaque ingrédient suffit pour comprendre toute la mécanique.

L'essentiel
Un neurone calcule f(w₁x₁ + … + wₙxₙ + b) : somme pondérée, plus biais, puis activation. Les poids et biais sont appris par l'entraînement (rétropropagation). Les couches successives permettent d'apprendre des motifs de plus en plus complexes.

Exemple — Un neurone artificiel calculé pas à pas (valeurs réelles)

```
import numpy as np

def sigmoide(z):
return 1 / (1 + np.exp(-z))

def neurone(x, w, b):
produits = x * w # contribution de chaque entrée
z = produits.sum() + b # somme pondérée + biais
return produits, z, sigmoide(z)

x = np.array([0.8, 0.5, 0.3]) # entrées (signaux normalisés)
w = np.array([0.4, -0.2, 0.9]) # poids « appris » par l'entraînement
b = -0.1 # biais
produits, z, sortie = neurone(x, w, b)
print("Produits :", produits.tolist()) # [0.32, -0.1, 0.27]
print("z =", round(z, 2)) # 0.39
print("Sigmoïde(z) =", round(sortie, 4)) # 0.5963
print("ReLU(z) =", max(0.0, z)) # 0.39

# Mêmes poids, autres entrées : le neurone change d'avis
p2, z2, s2 = neurone(np.array([0.1, 0.9, 0.2]), w, b)
print("Cas 2 : produits =", p2.tolist()) # [0.04, -0.18, 0.18]
print("Cas 2 : z =", round(z2, 2), "| sigmoïde =", round(s2, 3),
"| ReLU =", max(0.0, round(z2, 2)))
# -> z = -0.06 | sigmoïde = 0.485 | ReLU = 0.0
```

À retenir

- Les trois ingrédients d'un neurone : des poids, un biais, une fonction d'activation.
- La sigmoïde écrase n'importe quel score dans (0 ; 1) ; ReLU coupe les valeurs négatives à 0.
- Apprendre, pour un réseau, signifie ajuster ses poids pour réduire l'erreur mesurée sur les exemples.

Exercice

Un neurone reçoit les entrées x = (1,0 ; 0,6), les poids w = (0,5 ; −0,4) et le biais b = 0,2. 1) Calcule chaque produit, la somme pondérée z et la sortie sigmoïde (on donne : sigmoïde(0,46) ≈ 0,613 — vérifie seulement que ton z vaut bien 0,46). 2) Quelle serait la sortie avec ReLU ? 3) Si l'on augmente le poids de la deuxième entrée de −0,4 à +0,4, dans quel sens la sortie évolue-t-elle, et pourquoi ?

✔ Corrigé
1) Produits : 1,0 × 0,5 = 0,50 et 0,6 × (−0,4) = −0,24. Somme : 0,50 − 0,24 = 0,26 ; plus le biais : z = 0,26 + 0,2 = 0,46. Sortie sigmoïde ≈ 0,613 (valeur confirmée par exécution). Interprétation : si le seuil de décision est 0,5, ce neurone « vote » pour la classe 1. 2) ReLU(0,46) = 0,46 : la valeur positive passe inchangée. 3) Le produit de la deuxième entrée devient 0,6 × 0,4 = +0,24 au lieu de −0,24 : z augmente de 0,48 et passe à 0,94, la sigmoïde monte donc vers 0,72 environ — la sortie augmente. Moralité : le signe d'un poids détermine si l'entrée excite ou inhibe le neurone, et sa valeur absolue détermine avec quelle force.

Leçon 5 — Traitement automatique du langage naturel : initiation

Le traitement automatique du langage naturel (TAL, ou NLP en anglais) s'attaque à un problème fondamental : les algorithmes ne comprennent que des nombres, alors que le langage humain est fait de mots. Avant tout apprentissage, il faut donc transformer le texte en nombres. Cette transformation s'appelle la vectorisation, et elle commence par des étapes de nettoyage : passer en minuscules, découper le texte en mots (les jetons, ou tokens), parfois retirer les mots très fréquents et peu informatifs comme « le », « de », « un » — les mots vides — selon la tâche visée.

La méthode de vectorisation la plus simple est le sac de mots (bag of words) : on constitue le vocabulaire de tous les mots distincts du corpus, puis chaque document devient un vecteur de comptages — combien de fois chaque mot du vocabulaire y apparaît. L'exemple ci-dessous, compté à la main puis vérifié par le CountVectorizer de scikit-learn, montre trois phrases minuscules transformées en une table de 3 lignes et 8 colonnes. Le nom « sac » est honnête : la méthode jette les mots dans un sac et perd donc leur ordre. « Le chien mord l'homme » et « l'homme mord le chien » donnent le même vecteur, ce qui montre la limite de l'approche pour comprendre le sens — mais pour classer des documents par thème ou détecter des pourriels, compter les mots suffit souvent étonnamment bien.

Les méthodes modernes vont plus loin : TF-IDF pondère chaque mot par sa rareté dans le corpus (un mot présent partout pèse moins qu'un mot distinctif), et les plongements de mots (embeddings), appris par des réseaux de neurones, placent les mots de sens proche à des positions proches dans un espace de nombres — c'est le principe qui alimente les grands modèles de langage issus de l'architecture Transformer (2017). Retiens la chaîne complète, valable pour presque toutes les applications de TAL : texte brut → nettoyage et découpage → vectorisation → modèle d'apprentissage → prédiction. Chacune de ces étapes peut faire réussir ou échouer un projet.

L'essentiel
Le TAL transforme le texte en nombres avant tout apprentissage. Le sac de mots compte les occurrences de chaque mot du vocabulaire : simple et efficace pour classer, mais il ignore l'ordre des mots et donc une partie du sens.

Exemple — Sac de mots sur trois phrases : compté à la main, vérifié par scikit-learn

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
"le chat mange le poisson",
"le chien mange un os",
"le chat dort",
]
vec = CountVectorizer()
matrice = vec.fit_transform(documents).toarray()
print("Vocabulaire :", list(vec.get_feature_names_out()))
# -> ['chat', 'chien', 'dort', 'le', 'mange', 'os', 'poisson', 'un']
for i, ligne in enumerate(matrice):
print("Document", i + 1, ":", ligne.tolist())
# Document 1 : [1, 0, 0, 2, 1, 0, 1, 0] (le apparaît 2 fois)
# Document 2 : [0, 1, 0, 1, 1, 1, 0, 1]
# Document 3 : [1, 0, 1, 1, 0, 0, 0, 0]
# Vérification à la main, document 1 : chat=1, le=2, mange=1,
# poisson=1, les autres mots du vocabulaire = 0. C'est exact.
```

À retenir

- Pas de nombres, pas d'apprentissage : la vectorisation est l'étape obligatoire du TAL.
- Le sac de mots conserve les comptages, pas l'ordre : deux phrases aux mêmes mots ont le même vecteur.
- TF-IDF et les embeddings affinent l'idée : pondérer les mots rares, puis représenter le sens par des vecteurs appris.

Exercice

On ajoute un quatrième document au corpus de l'exemple : « le chien dort ». 1) Sans toucher au vocabulaire d'origine (les 8 mots), écris le vecteur du document 4. 2) Le vocabulaire changerait-il si l'on reconstruisait le sac de mots sur les 4 documents ? 3) Explique pourquoi les vecteurs des documents 2 et 4 se ressemblent, et ce que cette ressemblance permettrait à un classifieur.

✔ Corrigé
1) Document 4 = « le chien dort » : chat 0, chien 1, dort 1, le 1, mange 0, os 0, poisson 0, un 0 → [0, 1, 1, 1, 0, 0, 0, 0]. 2) Non : les quatre mots du document 4 (le, chien, dort) appartiennent déjà au vocabulaire ; aucun mot nouveau n'apparaît, le vocabulaire resterait de 8 mots. Si le document contenait un mot inédit, par exemple « souris », le vocabulaire passerait à 9 mots et tous les vecteurs gagneraient une colonne. 3) Documents 2 et 4 partagent « le », « chien » et « dort » ou « mange » : ils parlent tous deux du chien, leurs vecteurs ont des valeurs non nulles sur presque les mêmes positions. Un classifieur peut exploiter cette géométrie : des vecteurs proches dans l'espace des comptages correspondent souvent à des documents de même thème — c'est le principe qui permet de trier automatiquement des messages ou des articles.

Leçon 6 — Évaluer un modèle : exactitude, précision et rappel

Un modèle qui « marche » ne veut rien dire tant qu'on n'a pas mesuré comment il se trompe. Pour une classification, tout part de la matrice de confusion, un tableau qui croise la réalité et la prédiction. Pour un filtre anti-pourriel, on compte : les vrais positifs (VP), pourriels correctement bloqués ; les faux positifs (FP), messages normaux bloqués à tort ; les faux négatifs (FN), pourriels qui ont traversé le filtre ; les vrais négatifs (VN), messages normaux correctement livrés. Dans l'exemple exécuté ci-dessous, sur 20 messages dont 10 pourriels : VP = 8, FP = 3, FN = 2, VN = 7.

De ces quatre nombres naissent les métriques essentielles. L'exactitude (accuracy) est la proportion de prédictions correctes : (VP + VN) ÷ total = 15 ÷ 20 = 0,75. La précision répond à « quand le modèle dit positif, a-t-il raison ? » : VP ÷ (VP + FP) = 8 ÷ 11 ≈ 0,727. Le rappel (recall) répond à « parmi les vrais positifs, combien le modèle en attrape-t-il ? » : VP ÷ (VP + FN) = 8 ÷ 10 = 0,80. Le score F1 combine précision et rappel en une moyenne harmonique : ici 0,762. Pour le filtre, les 3 faux positifs sont des messages légitimes perdus — souvent plus graves que les 2 pourriels passés au travers : c'est la précision qu'il faudra surveiller en priorité.

L'exactitude seule est un piège classique. Si 95 messages sur 100 sont normaux, un « modèle » paresseux qui répond toujours « normal » affiche 95 % d'exactitude… avec un rappel de 0 sur les pourriels : il ne détecte rien du tout. C'est le problème des classes déséquilibrées, fréquent en fraude, en médecine et en maintenance. La règle professionnelle est donc : toujours regarder la matrice de confusion, choisir la métrique qui correspond au coût réel des erreurs (que risque-t-on le plus : rater un positif, ou alerter à tort ?), et se méfier des scores calculés sur de trop petits jeux de test — un point déjà croisé aux leçons 2 et que le projet de la leçon 7 illustrera concrètement.

L'essentiel
La matrice de confusion (VP, FP, FN, VN) est la source de toutes les métriques : exactitude = (VP+VN)/total, précision = VP/(VP+FP), rappel = VP/(VP+FN), F1 = moyenne harmonique des deux. Le choix de la métrique dépend du coût réel de chaque type d'erreur.

Exemple — Matrice de confusion d'un filtre anti-pourriel (métriques réelles calculées)

```
from sklearn.metrics import (confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score)

# Filtre testé sur 20 messages (1 = pourriel, 0 = message normal)
y_vrai = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
y_pred = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0]

tn, fp, fn, tp = confusion_matrix(y_vrai, y_pred).ravel()
print("Vrais positifs (VP) :", tp) # 8 pourriels bloqués
print("Faux positifs (FP) :", fp) # 3 messages normaux bloqués à tort
print("Faux négatifs (FN) :", fn) # 2 pourriels passés au travers
print("Vrais négatifs (VN) :", tn) # 7 messages normaux livrés

print("Exactitude = (VP+VN)/total =", round(accuracy_score(y_vrai, y_pred), 3))
# -> 0.75
print("Précision = VP/(VP+FP) =", round(precision_score(y_vrai, y_pred), 3))
# -> 0.727 (8/11)
print("Rappel = VP/(VP+FN) =", round(recall_score(y_vrai, y_pred), 3))
# -> 0.8 (8/10)
print("Score F1 =", round(f1_score(y_vrai, y_pred), 3))
# -> 0.762
```

À retenir

- Précision : « quand le modèle dit oui, a-t-il raison ? » — Rappel : « parmi les vrais oui, combien en trouve-t-il ? »
- Une exactitude élevée peut cacher un modèle inutile sur des classes déséquilibrées : toujours ouvrir la matrice de confusion.
- Il n'existe pas de « meilleure métrique » universelle : tout dépend du coût de chaque erreur dans le problème réel.

Exercice

Un filtre de commentaires toxiques est testé sur 100 commentaires : il bloque correctement 20 commentaires toxiques (VP = 20), bloque à tort 5 commentaires inoffensifs (FP = 5), laisse passer 10 commentaires toxiques (FN = 10) et laisse passer correctement 65 commentaires inoffensifs (VN = 65). Calcule l'exactitude, la précision, le rappel et le score F1. Puis dis quelle erreur te semble la plus grave pour un réseau social, et quelle métrique tu suivrais en priorité.

✔ Corrigé
Total = 20 + 5 + 10 + 65 = 100. Exactitude = (20 + 65) ÷ 100 = 0,85. Précision = VP ÷ (VP + FP) = 20 ÷ 25 = 0,80 : quand le filtre bloque, il a raison 4 fois sur 5. Rappel = VP ÷ (VP + FN) = 20 ÷ 30 ≈ 0,667 : un commentaire toxique sur trois passe au travers. F1 = 2 × 0,80 × 0,667 ÷ (0,80 + 0,667) ≈ 0,727 (valeurs vérifiées par le calcul). Discussion : laisser passer un commentaire toxique (FN) expose des utilisateurs à du harcèlement et peut avoir des conséquences humaines et légales ; bloquer à tort (FP) frustre un utilisateur innocent qui peut faire appel. Pour un réseau social, beaucoup d'équipes privilégient le rappel (ne pas laisser passer de contenu dangereux) tout en gardant un œil sur la précision pour ne pas censurer à tort — l'essentiel est de choisir explicitement, chiffres à l'appui, plutôt que de se contenter de l'exactitude flatteuse de 0,85.

Leçon 7 — Projet : construire et évaluer un modèle simple en Python

Ce projet assemble tout le cours en un programme complet et reproductible : prédire l'admission d'étudiants d'une promotion de l'IAM à partir de trois variables — heures de révision hebdomadaires, assiduité en cours et moyenne du semestre précédent. Le jeu de données contient 20 étudiants fabriqués pour l'exercice. La démarche suit le protocole professionnel : séparer les données en entraînement (14 étudiants) et test (6 étudiants) avec train_test_split en fixant random_state = 42 pour que le tirage soit reproductible, entraîner un arbre de décision limité à une profondeur de 3 (assez simple pour rester lisible), prédire sur le test, puis évaluer avec les métriques de la leçon 6.

Le résultat obtenu en exécutant le code est spectaculaire en apparence : exactitude, précision et rappel valent tous 1,0 sur le jeu de test — les 6 étudiants sont parfaitement classés, et la variable la plus importante selon l'arbre est l'assiduité (importance 0,74, contre 0,26 pour les heures et 0,00 pour la moyenne précédente). C'est ici que le praticien doit résister à l'euphorie : le jeu de test ne contient que 6 étudiants, donc chaque erreur vaudrait 17 points d'exactitude. En refaisant le même tirage avec d'autres graines aléatoires, le même arbre obtient 0,833 au lieu de 1,0 — mesuré, pas supposé. Un score parfait sur 6 exemples prouve que le pipeline fonctionne, pas que le modèle est parfait.

La dernière compétence du projet est donc le regard critique. Premier risque : le sur-apprentissage (overfitting), quand un modèle apprend par cœur le bruit des données d'entraînement au lieu de la structure générale. Sur un jeu bruité de 60 exemples dont 8 étiquettes volontairement erronées, un arbre sans limite de profondeur atteint une exactitude de 1,000 en entraînement mais seulement 0,722 en test : il a mémorisé les erreurs au lieu d'apprendre la règle. Deuxième risque : les données elles-mêmes — 20 étudiants d'une seule promotion ne représentent ni les autres filières ni les autres années. La conduite à tenir : limiter la complexité du modèle, mesurer sur des données jamais vues, augmenter le volume de données quand c'est possible, et toujours publier ses résultats avec la taille de l'échantillon. C'est cette honnêteté chiffrée qui distingue un projet d'IA crédible d'une démonstration

truquée.

L'essentiel
Un projet ML complet = données → séparation entraînement/test → entraînement → prédictions → évaluation → critique. Un score parfait sur un minuscule jeu de test n'est pas une preuve ; le sur-apprentissage guette dès que le modèle peut mémoriser au lieu de généraliser.

Exemple — Le projet complet : arbre de décision sur 20 étudiants (résultats réels)

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score

# 20 étudiants : (heures de révision, assiduité %, moyenne précédente)
X = np.array([
[2, 40, 8], [3, 55, 9], [4, 60, 10], [5, 70, 11], [6, 65, 12],
[7, 80, 13], [8, 85, 12], [9, 90, 14], [3, 80, 13], [4, 45, 8],
[5, 50, 9], [6, 75, 12], [7, 60, 10], [8, 95, 15], [2, 30, 7],
[5, 85, 13], [6, 55, 9], [9, 70, 12], [4, 90, 13], [7, 65, 11],
])
y = np.array([0, 0, 0, 1, 1, 1, 1, 1, 1, 0,
0, 1, 1, 1, 0, 1, 0, 1, 1, 1]) # 1 = admis

# 1) Séparation reproductible : 14 entraînement / 6 test
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
random_state=42)
# 2) Entraînement d'un arbre volontairement simple
modele = DecisionTreeClassifier(max_depth=3, random_state=42)
modele.fit(X_tr, y_tr)
# 3) Prédictions et évaluation sur des données jamais vues
pred = modele.predict(X_te)
print("Vrai :", y_te.tolist(), "| Prédit :", pred.tolist())
print("Exactitude test :", accuracy_score(y_te, pred)) # 1.0
print("Précision :", precision_score(y_te, pred),
"| Rappel :", recall_score(y_te, pred)) # 1.0 et 1.0
print("Importance des variables :",
np.round(modele.feature_importances_, 2))
# -> [0.26 0.74 0.00] : l'assiduité domine, la moyenne précédente
# n'est pas utilisée par cet arbre.
```

À retenir

- random_state fixé = tirage reproductible : n'importe qui peut refaire ton expérience et obtenir les mêmes chiffres.
- 6 exemples de test : chaque erreur vaut environ 17 points d'exactitude — les scores y sont forcément instables.
- Sur-apprentissage = entraînement parfait, test médiocre. Le remède : simplifier le modèle et évaluer sur du jamais-vu.

Exercice

1) Réexécute le projet en changeant uniquement la graine : train_test_split(..., random_state=1), puis random_state=7. Note les exactitudes obtenues (tu dois trouver 0,833 dans les deux cas). 2) Explique pourquoi le modèle n'a « pas changé » mais son score oui. 3) Un camarade propose de choisir la graine qui donne 1,0 et de la présenter comme LE résultat du projet. Est-ce honnête ? Que proposer à la place ?

✔ Corrigé
1) Avec les graines 1 et 7, l'arbre (même max_depth=3, même random_state interne) obtient 0,833, c'est-à-dire 5 étudiants bien classés sur 6 — valeurs mesurées lors de la préparation de ce cours, sur cinq tirages : 1,0 ; 0,833 ; 0,833 ; 1,0 ; 1,0. 2) Le modèle est le même algorithme avec les mêmes réglages, mais le tirage change la composition des jeux d'entraînement et de test : il suffit qu'un étudiant « limite » (par exemple très assidu mais révisant peu) tombe dans le test plutôt que dans l'entraînement pour faire basculer le score. Sur 6 exemples, un seul individu représente 1/6 du résultat. 3) Non, ce n'est pas honnête : choisir après coup le tirage le plus flatteur revient à maquiller la mesure. À la place, on annonce le protocole complet (taille des jeux, graine utilisée, et idéalement la moyenne des scores sur plusieurs tirages ou une validation croisée), de sorte que le lecteur puisse juger la robustesse du résultat. En IA comme ailleurs, un résultat qu'on ne peut pas reproduire ne vaut rien.

Travaux dirigés (TD)

TD 1 — Métriques d'un test de dépistage médical

Un modèle d'aide au diagnostic est évalué sur 200 patients. Il signale correctement 45 patients malades (VP = 45), déclenche 15 alertes à tort sur des patients sains (FP = 15), manque 5 patients malades (FN = 5) et rassure correctement 135 patients sains (VN = 135). 1) Construis la matrice de confusion. 2) Calcule l'exactitude, la précision, le rappel et le score F1, en écrivant chaque formule. 3) Un médecin te dit : « manquer un malade est bien plus grave qu'une alerte à tort ». Quelle métrique devient prioritaire ? Le modèle actuel est-il satisfaisant de ce point de vue ?

✔ Corrigé du TD 1
1) Matrice (lignes = réalité, colonnes = prédiction) : malade signalé 45, malade manqué 5 ; sain signalé à tort 15, sain rassuré 135. Total = 200. 2) Exactitude = (45 + 135) ÷ 200 = 180 ÷ 200 = 0,90. Précision = VP ÷ (VP + FP) = 45 ÷ 60 = 0,75. Rappel = VP ÷ (VP + FN) = 45 ÷ 50 = 0,90. F1 = 2 × 0,75 × 0,90 ÷ (0,75 + 0,90) = 1,35 ÷ 1,65 ≈ 0,818 (valeurs vérifiées par le calcul). 3) La métrique prioritaire est le rappel : il mesure la proportion de malades effectivement détectés. Ici, le rappel vaut 0,90 : 5 malades sur 50 passent entre les mailles. Selon la gravité de la maladie, ce peut être insuffisant — en dépistage, on cherche souvent un rappel de 0,95 ou plus, quitte à accepter davantage de fausses alertes (précision plus faible) qui seront levées par un examen complémentaire. Leçon générale : la « bonne » métrique se décide avec l'utilisateur final et le coût des erreurs, pas dans le code.

TD 2 — Comparer trois modèles sur le projet de la leçon 7

Sur le jeu des 20 étudiants de la leçon 7, on refait la séparation avec random_state=1 (toujours 14 étudiants en entraînement, 6 en test) et l'on compare trois modèles. Résultats mesurés : arbre de décision (profondeur 3) : exactitude test 0,833 ; KNN avec k = 1 : exactitude entraînement 1,000, exactitude test 0,833 ; KNN avec k = 5 : exactitude test 0,667. 1) Que vaut, en nombre d'étudiants, l'écart entre 0,833 et 0,667 sur ce jeu de test ? 2) Pourquoi le KNN avec k = 1 obtient-il toujours 1,000 en entraînement, et pourquoi ce chiffre ne prouve rien ? 3) Peut-on déclarer l'arbre « meilleur » que le KNN k = 5 ? Que faudrait-il faire pour trancher sérieusement ?

✔ Corrigé du TD 2

1) Le jeu de test compte 6 étudiants : 0,833 = 5/6 et 0,667 = 4/6. L'écart entre les deux modèles tient à UN seul étudiant. 2) Avec k = 1, le plus proche voisin d'un étudiant d'entraînement est lui-même : le modèle « prédit » donc parfaitement les données qu'il a sous les yeux — c'est de la mémorisation pure, l'extrême du sur-apprentissage. L'exactitude d'entraînement ne mesure pas la capacité à généraliser ; seule l'exactitude sur des données jamais vues compte. 3) Non : une différence d'un étudiant sur six ne permet aucune conclusion sérieuse (avec d'autres tirages mesurés pour ce cours, l'arbre est passé de 1,0 à 0,833 selon la graine). Pour trancher, il faudrait un jeu de test beaucoup plus grand, ou une validation croisée (plusieurs tirages dont on moyenne les scores), et comparer les modèles sur exactement les mêmes tirages. Retenir la règle : plus l'échantillon de test est petit, plus les classements de modèles sont fragiles.

Travaux pratiques (TP)

TP 1 — Refaire le projet, changer un paramètre à la fois

En t'appuyant sur le code du projet de la leçon 7 : 1) réexécute-le tel quel et vérifie que tu obtiens exactitude, précision et rappel de 1,0 ; 2) remplace l'arbre par un KNeighborsClassifier(k=3) et note ses scores sur le même tirage ; 3) reviens à l'arbre mais supprime la limite de profondeur (max_depth=None) et compare entraînement et test ; 4) change la graine du tirage (random_state=0, puis 123) avec l'arbre d'origine et note les scores. Rédige un compte rendu d'une page : tableau des essais, ce qui change, ce qui ne change pas, et ta conclusion sur la confiance à accorder à ces chiffres.

✔ Attendus du TP 1

Attendus : 1) le code d'origine redonne exactement 1,0 partout grâce au random_state fixé — sinon, vérifier que les données copiées sont identiques. 2 et 3) Les scores varient selon le modèle et la profondeur : sur des tirages de contrôle effectués pour ce cours, le même arbre obtient 1,0 (graines 42, 0 et 123) ou 0,833 (graines 1 et 7) — c'est la fourchette de référence pour juger les résultats de l'étudiant. 4) Le changement de graine suffit à faire bouger l'exactitude d'un cran de 1/6. Conclusion attendue dans le compte rendu : le pipeline est correct et reproductible, mais 6 exemples de test ne permettent pas de classer finement les modèles ; pour un vrai projet, il faudrait davantage de données et une validation croisée. Barème indicatif : reproduction fidèle 4 points, tableau des essais complet 6 points, comparaison entraînement/test (sur-apprentissage) 4 points, conclusion argumentée sur la taille d'échantillon 6 points.

Exercices corrigés supplémentaires

Exercice 1

Calcule à la main la sortie d'un neurone : entrées x = (1,0 ; 0,6), poids w = (0,5 ; −0,4), biais b = 0,2, activation sigmoïde (on donne sigmoïde(0,46) ≈ 0,613). Détaille produits, somme pondérée et conclusion si le seuil de décision est 0,5.

✔ Corrigé

Produits : 1,0 × 0,5 = 0,50 ; 0,6 × (−0,4) = −0,24. Somme pondérée : 0,50 − 0,24 + 0,2 = 0,46. Sigmoïde(0,46) ≈ 0,613 (vérifié par exécution). La sortie dépasse le seuil 0,5 : le neurone classe l'exemple dans la catégorie 1, avec une « confiance » modérée de 61 % — loin d'une certitude.

Exercice 2

Un filtre de commentaires toxiques obtient VP = 20, FP = 5, FN = 10, VN = 65 sur 100 commentaires. Calcule exactitude, précision, rappel et F1, puis explique en une phrase ce que chaque métrique raconte de ce filtre.

✔ Corrigé

Exactitude = (20 + 65) ÷ 100 = 0,85 : 85 % des décisions sont correctes. Précision = 20 ÷ (20 + 5) = 0,80 : quand le filtre bloque, il a raison 4 fois sur 5. Rappel = 20 ÷ (20 + 10) ≈ 0,667 : il échappe un commentaire toxique sur trois. F1 = 2 × 0,80 × 0,667 ÷ (0,80 + 0,667) ≈ 0,727 (calcul vérifié). Lecture d'ensemble : le filtre est plutôt fiable quand il bloque, mais son maillage est trop large — c'est le rappel qu'il faut améliorer pour protéger les utilisateurs.

Exercice 3

Fais tourner K-Means à la main sur les points A(1, 1), B(2, 1), C(8, 9), D(9, 8), avec K = 2 et les centres initiaux A et D. Donne les groupes et les centres après convergence.

✔ Corrigé

Itération 1 : A et B se rattachent au centre A, C et D au centre D. Nouveaux centres : moyenne de {A, B} = (1,5 ; 1,0) ; moyenne de {C, D} = (8,5 ; 8,5). Itération 2 : les affectations ne changent pas avec ces centres, donc les centres non plus — convergence (calcul confirmé par exécution). Groupes finaux : {A, B} et {C, D}, de centres (1,5 ; 1,0) et (8,5 ; 8,5).

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. L'atelier fondateur où le terme « intelligence artificielle » a été proposé s'est tenu à Dartmouth en…

A) 1950

B) 1956

C) 2012

2. Un problème de régression est un problème où l'on prédit…

A) Un nombre continu, comme un prix ou une note

B) Une catégorie, comme « pourriel » ou « normal »

C) Un mot de vocabulaire

3. Dans l'apprentissage supervisé, les données d'entraînement sont…

A) Sans aucune réponse associée

B) Triées par ordre alphabétique

C) Accompagnées de leurs étiquettes (les bonnes réponses)

4. Dans l'algorithme K-Means, le nombre K de groupes est…

A) Deviné automatiquement par l'algorithme

B) Choisi par l'humain avant de lancer l'algorithme

C) Toujours égal à 3

5. Un neurone artificiel calcule sa sortie en appliquant une fonction d'activation à…

A) La somme des produits entrées × poids, plus le biais

B) La moyenne des étiquettes

C) La plus grande des entrées

6. La fonction sigmoïde transforme n'importe quel nombre réel en une valeur…

A) Toujours négative

B) Comprise entre 0 et 1

C) Toujours supérieure à 1

7. La méthode du sac de mots a pour principale limite…

A) Qu'elle perd l'ordre des mots dans le document

B) Qu'elle ne fonctionne qu'en anglais

C) Qu'elle exige une carte graphique

8. La précision d'un classifieur se calcule par la formule…

A) VP ÷ (VP + FN)

B) VP ÷ (VP + FP)

C) VP ÷ total

9. Un modèle qui répond toujours « message normal » sur un flux composé à 95 % de messages normaux obtient une exactitude de 95 %, mais il est inutile car…

A) Son rappel sur les pourriels vaut 0 : il ne détecte rien

B) Son temps de calcul est trop long

C) Il n'utilise pas de réseau de neurones

10. Le sur-apprentissage (overfitting) se reconnaît quand…

A) Le modèle est trop lent à entraîner

B) Le modèle est excellent sur les données d'entraînement mais médiocre sur des données nouvelles

C) Le modèle refuse de répondre

Corrigés du QCM

1. Réponse B — L'atelier de Dartmouth (1956) marque la naissance officielle du domaine ; 1950 est l'année de l'article de Turing, 2012 celle du succès d'AlexNet.

2. Réponse A — La régression prédit une valeur numérique continue ; prédire une catégorie s'appelle une classification.

3. Réponse C — Supervisé signifie qu'un « corrigé » accompagne les exemples : ce sont les étiquettes que le modèle apprend à prédire.

4. Réponse B — K-Means exige qu'on lui donne K ; le choix se fait par essais (inertie) et par bon sens métier.

5. Réponse A — Sortie = $f(w_1x_1 + \dots + w_nx_n + b)$: somme pondérée, biais, puis activation comme la sigmoïde ou ReLU.

6. Réponse B — La sigmoïde écrase les scores dans l'intervalle (0 ; 1), ce qui permet de les lire comme des probabilités.

7. Réponse A — Deux phrases contenant les mêmes mots dans un autre ordre ont le même vecteur : le sac de mots compte, il ne lit pas.

8. Réponse B — Précision = VP/(VP+FP) : parmi les positifs annoncés, quelle proportion est correcte. VP/(VP+FN) est le rappel.

9. Réponse A — Exactitude flatteuse, détection nulle : c'est le piège des classes déséquilibrées, démasqué par la matrice de confusion.

10. Réponse B — Le modèle a mémorisé les exemples (y compris leurs erreurs) au lieu d'apprendre la structure générale : l'écart entraînement/test le trahit.

Résumé — les notions importantes

• L'IA naît en 1956 à Dartmouth, connaît deux hivers, puis triomphe après 2012 grâce aux données massives, aux cartes graphiques et à des algorithmes d'apprentissage mûris.

• Apprentissage supervisé : régression si la cible est un nombre (on rapporte par exemple un R^2), classification si c'est une catégorie (on rapporte des métriques de confusion).

• On n'évalue jamais un modèle sur ses données d'entraînement : le jeu de test reste de côté jusqu'au bout, et sa taille conditionne la confiance dans les scores.

• Le clustering regroupe sans étiquettes ; K-Means alterne affectation au centre le plus proche et recentrage au barycentre, avec un K choisi par l'humain.

• Un neurone calcule une somme pondérée de ses entrées, ajoute un biais et applique une activation (sigmoïde, ReLU) ; les réseaux empilent ces neurones en couches et apprennent en ajustant les poids.

• Le traitement du langage commence par vectoriser le texte : le sac de mots compte les occurrences, simple et utile, mais il ignore l'ordre des mots.

• Exactitude, précision = VP/(VP+FP), rappel = VP/(VP+FN) et F1 se lisent dans la matrice de confusion ; la métrique prioritaire dépend du coût réel de chaque erreur, et tout score parfait sur un minuscule échantillon doit éveiller la méfiance.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Histoire et grands domaines de l'intelligence artificielle
1956, Dartmouth : naissance officielle de l'IA ; 1950, Turing avait déjà posé la question fondatrice. ; Deux hivers de l'IA montrent le danger des promesses excessives : un domaine se juge à ses résultats mesurés. ; Le deep learning triomphe après 2012 grâce aux données massives et aux cartes graphiques, pas grâce à une seule idée géniale.

Fiche 2 — Apprentissage supervisé : régression et classification
Régression = prédire un nombre ; classification = prédire une catégorie. Le type de la cible décide de la famille. ; fit() entraîne, predict() prédit, score() évalue : c'est le cycle de tous les modèles scikit-learn. ; Un R^2 très élevé sur un exemple fabriqué ne prouve rien sur le terrain : seules comptent les performances sur des données nouvelles.

Fiche 3 — Apprentissage non supervisé : le clustering
K-Means : affecter chaque point au centre le plus proche, recalculer les centres, répéter jusqu'à convergence. ; Le nombre K de groupes est un choix humain, éclairé par l'inertie et par le bon sens métier — pas une vérité automatique. ; Un numéro de cluster n'est pas une étiquette : nommer les groupes fait partie du travail d'analyse.

Fiche 4 — Réseaux de neurones : principes de fonctionnement
Les trois ingrédients d'un neurone : des poids, un biais, une fonction d'activation. ; La sigmoïde écrase n'importe quel score dans (0 ; 1) ; ReLU coupe les valeurs négatives à 0. ; Apprendre, pour un réseau, signifie ajuster ses poids pour réduire l'erreur mesurée sur les exemples.

Fiche 5 — Traitement automatique du langage naturel : initiation
Pas de nombres, pas d'apprentissage : la vectorisation est l'étape obligatoire du TAL. ; Le sac de mots conserve les comptages, pas l'ordre : deux phrases aux mêmes mots ont le même vecteur. ; TF-IDF et les embeddings affinent l'idée : pondérer les mots rares, puis représenter le sens par des vecteurs appris.

Fiche 6 — Évaluer un modèle : exactitude, précision et rappel
Précision : « quand le modèle dit oui, a-t-il raison ? » — Rappel : « parmi les vrais oui, combien en trouve-t-il ? » ; Une exactitude élevée peut cacher un modèle inutile sur des classes déséquilibrées : toujours ouvrir la matrice de confusion. ; Il n'existe pas de « meilleure métrique » universelle : tout dépend du coût de chaque erreur dans le problème réel.

Fiche 7 — Projet : construire et évaluer un modèle simple en Python
random_state fixé = tirage reproductible : n'importe qui peut refaire ton expérience et obtenir les mêmes chiffres. ; 6 exemples de test : chaque erreur vaut environ 17 points d'exactitude — les scores y sont forcément instables. ; Sur-apprentissage = entraînement parfait, test médiocre. Le remède : simplifier le modèle et évaluer sur du jamais-vu.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — C'EST QUOI L'IA, EN VRAI ? #ia #simple #llm

Chaîne : IA tout simplement · Langue : Français · Niveau : Débutant
Une explication sans jargon de ce qu'est vraiment l'IA d'aujourd'hui, de la différence avec un programme classique et de la façon dont elle apprend par exemples. Parfaite pour ouvrir la leçon 1 avant le voyage historique.

https://www.youtube.com/watch?v=g4OZs3UqQ-A

Leçon 2 — Le Machine Learning Expliqué Simplement: les Étapes de Construction d'un Modèle

Chaîne : LeCoinStat · Langue : Français · Niveau : Débutant à intermédiaire
La démarche complète de construction d'un modèle de machine learning, de la définition du problème à la validation, en passant par la collecte et la préparation des données. Elle recouvre exactement le protocole suivi dans les leçons 2 et 7.

https://www.youtube.com/watch?v=AKNJc-xC42w

Leçon 4 — RÉSEAU DE NEURONES (2 COUCHES) - DEEP LEARNING 7

Chaîne : Machine Learnia · Langue : Français · Niveau : Intermédiaire
Comment plusieurs neurones s'organisent en couches et comment l'information avance dans le réseau (propagation avant), puis comment l'entraînement corrige les poids (rétropropagation). Le prolongement direct du calcul de neurone de la leçon 4.

https://www.youtube.com/watch?v=YMP-lU-xqyc

Leçon 4 — Réseau de neurones : pourquoi un seul neurone ne suffit pas ? — Cours 6/11

Chaîne : Formation · Langue : Français · Niveau : Débutant à intermédiaire

Pourquoi un neurone isolé atteint vite ses limites et comment les couches de neurones se répartissent la détection de motifs, avec un exemple pratique en Python. À regarder après avoir fait le calcul à la main de la leçon 4.

https://www.youtube.com/watch?v=wYF9JPHrnKI

Leçon 7 — PYTHON SKLEARN: KNN, LinearRegression et SUPERVISED LEARNING (20/30)

Chaîne : Machine Learnia · Langue : Français · Niveau : Intermédiaire

L'API uniforme de scikit-learn (instancier, fit, predict, score) appliquée à la régression linéaire et aux plus proches voisins. Idéal pour refaire le projet de la leçon 7 en variant les modèles comme le demande le TP.

https://www.youtube.com/watch?v=P6kSc3qVph0

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Russell & Norvig — « Intelligence artificielle » (Pearson) : le manuel de référence mondial, des fondements historiques aux méthodes modernes.

• Documentation officielle de scikit-learn (scikit-learn.org) — guides de l'utilisateur, exemples exécutables et référence complète de l'API.

• A. Géron — « Machine Learning avec Scikit-Learn, Keras et TensorFlow » (O'Reilly) : un parcours très pratique, proche de l'esprit de ce cours.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés via YouTube à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.